Assessed Coursework 2 — Systems Programming

This coursework specification, and the example code provided during the course, is Copyright 2026 Heriot-Watt University. Distributing this coursework specification or your solution to it outside the university is academic misconduct and a violation of copyright law.

1 Overview

The aim of this coursework is to develop a simple, systems-level application in C and ARM Assembler, running on a Raspberry Pi with attached devices.

The learning objective of this coursework is for students to obtain detailed understanding of the interaction between embedded hardware and external devices, in order to control this interaction in low-level code. The programming skills will cover detailed resource management and time sensitive operations. Design choices regarding languages, tools, and libraries chosen for the implementation need to be justified in the accompanying report. This coursework will develop personal abilities in articulating system-level operations and identifying performance implications of given systems through the written report that should accompany the complete implementation.

The report needs to critically reflect on the software development process for (embedded) systems programming, contrast it to main stream programming, and comment on performance-relevant design choices as well as impact on resource consumption.

This CW must be done in pairs, using group-sets in Canvas. Team up with a fellow student on the course and sign-up to one group on Canvas under `People -> Group`.

2 Lab Environment

Hardware environment: As hardware platform, a Raspberry Pi 2, 3 or 4 (**not 5**) with the starter kit introduced in the course should be used. The format of hand-out for the Raspberry Pi kits will be specified as Canvas announcement specific to the campus. At Edinburgh, the Raspberry Pi can be directly connected to the machines in the Linux Lab (EM 2.50) using a KVM switch. See the Lecture slides ("Course Overview") and the technical HOWTOs (Canvas) on this topic. You can then use the keyboard and mouse for input and the monitor for output and work as if on a standard Linux machine. **For other campuses, follow the instructions on Canvas, for campus-specific setup.**

Software environment: The SD card that is part of the starter kit uses Raspberry Pi OS (32-bit) as Linux-based operating system. It is recommended that you stick to this version, although there might be a more recent version of Raspberry Pi OS available. If you are installing Raspberry Pi OS yourself, make sure that you are using the **32-bit version**, otherwise assembler code will not work as expected.

For developing the code you should use the standard GNU toolchain (`gcc`, `as`, `ld`) that comes with the version of Raspberry Pi OS that is installed on the SD card. It is **strongly recommended** that you use the GNU debugger (`gdb`) for debugging your code. To check for and debug memory errors, we recommend that you use GCC's Address Sanitizer. See the lecture slides in the Programming Languages track of the course.

3 Embedded Systems Programming: pinCrack Application

In this assignment, you are required to implement a simple application (app) of cracking a PIN (Personal Identification Number), as used for example with bank cards. You will build a device that makes guesses at a secret PIN, based on information about how close the guessed PIN is to the correct PIN.

The application needs to run on an Raspberry Pi 2, 3 or 4 (**not** 5), with the following attached devices: two LEDs, a button, and an LCD display (with attached potentiometer for controlling contrast). The devices should be connected to the RPi via a breadboard, using the RPi kit that was handed out early in the course. See the section "Wiring" and the Fritzing diagram below.

Application: For `pinCrack` we assume that an attacker is trying to guess a PIN and can use information on how close a guessed sequence is to the secret sequence. The attacker inputs a sequence of numbers using the button as an input device. The RPi should then calculate and return (via LED blinks) the Hamming distance (see definition below) between the input sequence (PIN) and the secret sequence (PIN). Finally, the RPi should search all possible values to find the secret sequence (PIN), again using information provided by the Hamming distance.

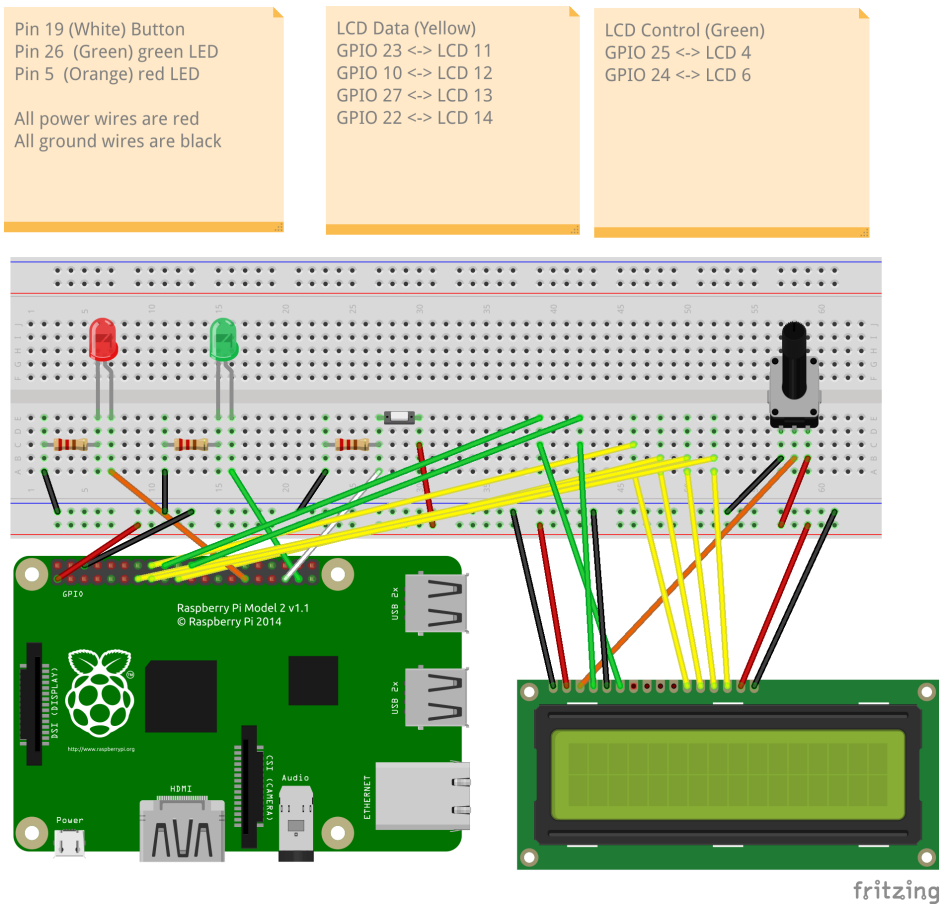
The objective of this CW is to (1) build a RPi configuration (wiring) that allows input of a sequence of numbers using a button, and communicating through LEDs and an LCD display; (2) implement cracking code that uses the input sequence with information from the Hamming distance to efficiently crack/find the secret PIN. The CW is split into tasks, that refine the functionality of the second (cracking) objective. A submission should identify and demonstrate the most advanced task implemented (which subsumes earlier tasks).

Coding: The application should be developed on the Raspberry Pi 2, 3 or 4 (**not** 5), using the device configuration below, with inlined ARM assembler code to directly control the attached devices through GPIO connections. No external libraries for programming the devices should be used in the final application. For Tasks 1–4 of the application, the *sequence length should be fixed at 3 (SEQL=3)* and *the possible numbers to use at each position should be fixed at 3 (DIGITS=3)* as well. In a "debug" mode the program should print the secret PIN at the beginning, so that the answers given can be checked.

Wiring/Devices: Two LEDs should be used as output devices: one (green) LED for data, and another (red) LED for control information (e.g. to separate parts of the input and to start a new round). The **green data LED (right)** should be connected to the RPi using **GPIO pin 26**. The **red control LED (left)** should be connected to the RPi using **GPIO pin 5**. A **button** should be used as input device. It should be connected to **GPIO pin 19**. An LCD should be used as an additional output device. It should be connected as follows:

LCD	GPIO	LCD	GPIO
1	(GND)	9	(unused)
2	(3.3V Power)	10	(unused)
3	(Potentiometer)	11 (DATA4)	23
4 (RS)	25	12 (DATA5)	10
5 (RW)	(GND)	13 (DATA6)	27
6 (EN)	24	14 (DATA7)	22
7	(unused)	15 (LED+)	(3.3V Power)
8	(unused)	16 (LED-)	(GND)

This means that the 4 data connections to the LCD display are connected to these 4 GPIO pins on the RPi: 23, 10, 27, 22. All devices should be connected to the RPi using a breadboard. Below is a **Fritzing diagram** that visualises the entire wiring: Note that the *middle* pin of the potentiometer needs to be wired to LCD 3, and the other 2 legs to ground and power as shown in this version. Make sure you are using the 3.3V power output from the Raspberry Pi, not the 5V power output. The Raspberry Pi inputs may be damaged if you connect them to 5V. Note that the potentiometer is providing a variable voltage to the LCD display allowing you to adjust the contrast on the display. You might need to adjust contrast to make text shown on the display easily readable. You can connect the RPi GPIO pins directly to the LCD pins, or plug the LCD directly into the breadboard, but it's usually easier to use a breadboard as shown.



LCD display: Follow the information in the diagram above. To get a closer look at the wiring, you can (but don't have to) download the project file of the above Fritzing diagram application here. You can view it using the Fritzing diagram application. Follow this FAQ on how to download and install the Fritzing diagram application. The main advantage of viewing the diagram inside the application is that you can zoom in to examine the wiring, and you get info on the pin numbers when you hover the mouse over them. This is optional, and you can work just with the diagram above, and the pin numbers shown in the diagram.

Sample code for the control of LED, button and LCD can be found in the sample sources section of the Course Information page for F28HS.

Note that the low-level operations of setting the mode of a pin, writing to an LED or LCD, and reading from a button need to be encoded in inline ARM Assembler, embedded into a C

function. Empty definitions for these functions `digital_write`, `pin_mode`, `read_button`, are given in the file `lcd-binary.c` in the gitlab repo, and the code for these functions needs to be completed. **Coding style requirement:** The code for these functions needs to use pointer arithmetic to access GPIO registers and the GPIO base address needs to be held in register `R2`. The **Hamming distance function also needs to be implemented in ARM Assembler, in a separate file using the ARM procedure call standard** (see discussion below). The rest should be done in C, and you can draw on the sample code to build the functionality of the application.

It is strongly recommended that you do the lab sheets for LED, LCD and button control, before starting the CW. However, you may have to adapt the wiring used in the lab-sheets to the one prescribed in the CW spec.

Task 1: Greeting: For this initial task you first have to construct the wiring as described above. The application should start with a greeting to the user as follows:

1. At start the RPi should show a personalised greeting, based on the student's family name: have the RPi blink one red for every consonant and 1 green for every vowel in your surname (first 5 letters only). For example, for the surname "Smith" expect this sequence of LED blinks (at each time only one colour should be on): red, red, green, red, red.
2. On the LCD display, show the first 5 letters of your surname.

Task 2: Input of a sequence using a button: For this task you use the wiring, as described above, and develop C code to input a sequence of numbers (representing a PIN) using button presses and to communicate the result using LEDs and an LCD display:

1. The Raspberry Pi should act as **codekeeper**, so needs to generate a (secret) random sequence and store it in the variable `theSeq` (code given in `initSeq` of starter code).
2. A number is entered using the button as an input device. Each number is entered as the number of button presses, i.e. press twice for an input of two etc.
3. While the button is pressed, the green LED should be on.
4. A fixed time-out should be used to separate the input of successive numbers. Use interval timers as introduced in the lectures.
5. The red control LED should blink once to acknowledge input of a number.
6. Then the green data LED should repeat the input number by blinking as many times as the button has been pressed.
7. Repeat these steps of button-input and LED-echo for each element of the input sequence (representing a PIN).
8. Once all values have been entered and echoed, the red control LED should blink two times to indicate the end of the input.

Reminder: For this task the length of the sequence is fixed to 3 and the possible numbers as 1–3. This is fixed through the constants `SEQL` and `DIGITS` in the file `cw2-config.h` of the starter code. In later tasks, this will be made parameterisable.

Coding style requirement: The input sequence needs to be gathered in an array of integers with the name `attemptSeq`. You need to dynamically allocate the array and populate the values in the loop of reading button presses. Remember to treat errors in allocation and values out-of-range appropriately.

Task 3: Computing the Hamming distance: This task continues with the output from Task 2 in the form of an input sequence, entered through button presses, and a secret sequence, generated by the RPi at the beginning. In this task the **Hamming distance** (see below) of both sequences should be calculated and displayed.

1. After completing the input of a sequence (as in Task 2), the app should calculate, the Hamming distance between the two sequences: the input sequence and the secret sequence (see below).
2. The app should print the Hamming distance on the terminal and additionally show it on the LCD display.
3. This behaviour should be triggered by using the command-line option `-u` (for unit testing; see Command-line Usage below).
4. If the options `-s` and `-r` are used together with `-u`, the corresponding numbers should be used as secret PIN and input PIN, respectively. If both are provided, no input of a sequence of numbers is necessary.

Hamming distance (in ARM Assembler): The **Hamming distance** between two sequences is the number of positions in which the two sequences differ. In this case the sequences are arrays of integer values. As a visual example for the two sequences (you do NOT need to reproduce this output)

```
1 1 2 2
1 2 1 2
-------
= + + =
```

the differing positions are marked as `+` and thus the Hamming distance should be computed as 2. For detail see this Wikipedia page on the Hamming distance (or see alternative location here). The task in this part of the application is to implement an **ARM Assembler function** that computes the Hamming distance over two arrays of integers of given (same) length. This code should be implemented as an ARM Assembler sub-routine, matching ARM sub-routine calling conventions (AAPCS). It should be implemented in a separate Assembler file (`hamming.s`), and be used from the C code in the main file. The prototype for the function should be:

```
int hamming(const int *x, const int *y, int n);
```

where `x,y` point to the start of the arrays to compare, and `n` is the length.

Hint: It is recommended that you first develop an implementation of this function in C, and then translate this code into (external) assembler code, that can be linked with the main program. You'll get partial marks if there is only a C implementation, full marks for a fully functioning ARM Assembler implementation.

Task 4: Naive cracking using Hamming distance: This task continues with the output from Task 2 (input sequence, entered through button presses, and the secret sequence, generated by the RPi at the beginning), and builds on the implementation of the Hamming distance from Task 3. In this task, we assume that the attacker has access to the **Hamming distance** (see above) between both sequences, and can use this information to narrow down a brute-force search of trying every possible PIN. In the starter code this is captured by the `submit_PIN` function, which simulates the submission of an input PIN for testing against the secret PIN. In a real application, testing whether a guessed PIN is correct

would take a significant amount of time. The `submit_PIN` function simulates this by performing a sleep (do nothing) operation for a customisable time period (customise with `-S` option). This task should proceed in the following steps:

1. The app should calculate, and print the Hamming distance between the two PINs (sequences), the input PIN and the secret PIN (see above). This step is the same as in Task 4, but in this task we continue with a search phase in the next steps.
2. Implement a *brute-force search* of all possible PINs, but call `submit_PIN` only for those PINs with a matching Hamming distance from the input PIN, i.e. the same Hamming distance as the one between the input PIN and the secret PIN.
3. Print (on the terminal) the PIN found by this search and confirm it is the secret PIN.
4. Print (on the terminal) the runtime of the search and the number of calls to `submit_PIN`.
5. Blink the green LED twice if a result has been found.
6. LCD display: show the message "PIN found" and the sequence on the LCD display, if a result has been found.
7. If the command-line option `-e` is used (see Command-line Usage below), the search should not finish when finding the secret PIN, but rather continue with testing all remaining PINs. This is useful for deterministically evaluating the performance of your search.

Task 5: Extend both the input and the cracking phase to handle sequences of arbitrary length: This task should achieve the same functionality as the previous tasks (PIN input and PIN cracking), but should be coded in such a way that the *sequence length and the number of digits can be specified at runtime* using the `-n` and `-m` options, respectively. For stand-alone testing, the `-s` and `-r` options should be supported to read the secret sequence and the input (reference) sequence from the command-line, respectively (see Command-line Usage below).

Whereas a brute-force search, that can be used for Task 4, can be implemented with nested for-loops, the iteration over an array of arbitrary length needs to use a different structure.

Code structure: You should enumerate all PINs that have a matching Hamming distance (i.e. the same Hamming distance as the one between the input PIN and the secret PIN), and check only these PINs using the `submit_PIN` function. To achieve this use a two level design (typically implemented as two nested loops).

The *outer loop* should select all possible positions in the reference sequence that can be modified. For example, if the Hamming distance is 2, you will need to modify 2 positions in the reference (input) sequence to find the secret PIN. The outer loop would, in this case, enumerate all possible choices for these two positions.

The *inner loop* takes the information about the positions that should be changed, and enumerates all sequences that can be generated by replacing the values in these positions. In the body of the inner loop you should call the `submit_PIN` function and count the number of submits.

Enumerating all possible PINs means generating each PIN one-by-one. You do NOT need to build a data structure that captures all of them. You should report the total number of submits (in the variable `submits`) and the position in the search at which the secret sequence was found (variable `found_at`). When using the `-e` command-line option, the search should be exhaustive, i.e. not finish after finding the secret sequence but continue to enumerate all possible PINs with a matching Hamming distance.

Document your design choices for code structure and how to do the iteration in your report. In the report, reflect on the performance of your implementation, optimisations taken, and opportunities for further optimisations. In this discussion you should reflect on architectural concepts that we have covered in the lectures and that can increase the performance of such an implementation.

Note: For a longer sequence length the search will be very time consuming. Test this task with a *sequence length of 6 and a max-value of 12* (`-n 6 -m 12`). An exhaustive search with these settings should still be feasible (depending on the Hamming distance). Specifically, test and document (in the report) the run with these command-line options (you might need to prefix the line with `sudo`):

```
./cw2 -e -n 6 -m 12 -s 123123 -r 111123
```

Command-line usage: The application shall be named `cw2` and provide a command-line interface to test its functionality like this:

```
./cw2 [-d] [-v] [-e] [-l] [-S <delay>] [-n <seqlen>] [-m <maxval>]
      [-u] [-s <secret sequence>] [-r <reference sequence>]
```

If run without any options, the program should show the behaviour specified above (Tasks 1–5). If run with the `-d` option the app should run in debug mode, and show the secret sequence, the guessed sequence and their Hamming distance, and then continue with the search. If run with the `-l` option, the app should only print a message to the LCD display (for testing only). If run with the `-s` option, the `<secret sequence>` should be used as the sequence to guess (this is useful in combination with the `-d` option to debug the program). If run with the `-r` option, the `<reference sequence>` should be used instead of an input sequence (in this case no button input should be expected and the app should continue with the search; see comments in Task 5). If run with the `-e` option, the program should perform an *exhaustive search* i.e. trying all PINs (but still submit only those with correct Hamming distance between test PIN and reference PIN) rather than stop on finding the secret PIN. The `-n` option should specify the sequence length (`<seqlen>`). The `-m` option should specify the maximum value for a number (`maxval`) in the sequence (i.e. values 1 to `maxval` are permitted as input). The `-S` option should specify the submit delay in micro-seconds (see above). If run with the `-u` option, the app should run a *unit test* on 2 input sequences, specified through options `-s` and `-r`, and print the Hamming distance between both sequences., e.g.

```
./cw2 -u -s 112 -r 121
```

should print (on the terminal)

```
Hamming distance:
2
```

Note: The processing of these options is done in the starter code provided via the gitlab-student repo. You just need to make sure to use the correct variables as indicated in the comments of the code. Study the comments provided at the beginning of the `main` function in the starter code.

GitLab repository: To start your project fork the following GitLab repository, and then clone your forked copy onto a local machine. Ideally, do this directly on the Raspberry Pi. If you don't have a

network connection, clone onto a local machine and then transfer the files to the Raspberry Pi, using for example a USB stick. Remember, you can develop the C parts, including unit testing for the Hamming distance, on any Linux machine. The GitLab repo contains template code with a *suggested* structure for the application as a C program in `cw2.c`. Check the comments in these files as well as the top-level `README.md` file in the GitLab repository.

Testing: Test the program logic of the application by running the program (in debug mode, which shows the secret sequence at the beginning) with a given secret sequence (using the `-s` option) and a reference sequence (using the `-r` option). Add the output for this run as screenshot in the appendix of the report. Also, run a unit-testing setup (using the `-u` option) with the sequences `112` and `121` and show the result (which should be the Hamming distance): `./cw2 -e -u -s 112 -r 121`

4 Submission

You must submit the complete project files, containing the source code, the compiled cw2 executable, and the report (in `.pdf` format) as one `.zip` file. Submission is through the Coursework 2 assignment on Canvas. Please check the assignment details on Canvas to see your submission deadline — different campuses may have different deadlines. The main executable driving the application should be called `cw2`, as discussed in "Command-line Usage" above. **This coursework is worth 60% of the module's mark (including 6% for the assessed lab).**

The submitted coursework needs to be **demonstrated**. Demonstration is **mandatory** and no marks will be awarded to submissions that have not been demonstrated. Note that the demo is another opportunity to receive feedback on your code. Demonstration is either live in a lab session, or per recorded video. The format will be specified by the local course leader.

You are marked for the functionality of the application, but also for code quality and the discussion in the short report. The marking scheme for this project is attached. **This project must be done in pairs.** Following the submission, there might be selected demos for submissions that don't sufficiently explain behaviour in the report.

Late Submission Policy

The standard **penalty of -30%** of the maximum available mark applies to late submissions. No submissions will be accepted after 5 working days beyond the submission deadline.

5 Report Format

The report should have about 3 – 5 pages and needs to cover the following:

- The hardware specification and wiring (for LEDs and Button) that is used as hardware platform.
- A description of how the work was divided between the members of the group, indicating what each student was responsible for.
- A short discussion of the code structure, specifying the functionality of the main functions.
- A discussion of performance-relevant design decisions, and implications on resource consumption.

- A list of functions directly accessing the hardware (for LEDs, Button, and LCD display) and which parts of the function use assembler and which use C.
- The name and an interface discussion (what are the inputs, what is the output of the sub-routine) of the Hamming function implemented in ARM Assembler.
- An example set of output from the application.
- A summary, covering what tasks have been achieved (and what not), outstanding features, and what you have learnt from this coursework.

Marking Scheme

Criteria	Marks
Meeting system requirements and functionality (as specified in Section 3) Indicative split of marks: T1: greetings (2), T2: input w/ timeout (6) T2: LED & Button in Asm (8), T3: Hamming fct in Asm (6), T4: naive search using Hamming distance (2), T5: extended search (6) rest (cmd-line, perf etc) (5)	35
Report Quality Contents matching the structure in Section 5; discussion of program logic and of the core functions, controlling the GPIO interface; summary of learning outcomes achieved.	6
The Application Code quality (both C and ARM Assembler), clear function interfaces, sufficient comments.	8
Initiative, innovation, professionalism and efforts above & beyond the call of duty	5
Assessed Lab The implementation of the “Traffic Lights” lab needs to be demo’ed in a lab slot.	6
Total marks	60

Generative AI Usage Policy

Our policy on the use of generative AI tools in F28HS is as follows:

- Where English is not your first language you may use GenAI to translate the assessment instructions or other relevant content. You may not use GenAI to translate your assessment answers - you must write them in English, as HW requires all work to be completed in English and to be your own work.
- **No GenAI generated content is permitted in your assessment submission.**
- You should ensure that any use of GenAI to support your assessment planning is identified in your notes to ensure that you do not unintentionally use it in the assessment.
- This policy corresponds to Level 1 of the University GenAI usage levels

This covers your use of tools such as ChatGPT, GitHub Copilot and other “AI search” or “code completion” tools. You must not use these tools to generate text or code for F28HS. We will report suspected use of generative AI as academic misconduct.

Deadline: see Canvas module

Academic Misconduct

Your attention is drawn to the university policy on academic misconduct and to the Contract Cheating Risk and Avoidance Guide (available on Canvas).

This is a **pair project** and you will have to identify the contributions of each group member in the report. Each group member needs to contribute a substantial implementation task to the project.

The code you submit must be the work of either of the two group members. If some text or code in the coursework has been taken from other sources (beyond the material provided for this course), you must give a clear reference to the source in comments in the code or the report. We cannot give any marks for work that is not your own.

Failure to clearly reference work that has been obtained from other sources, or copying the words or code of another student (other than the group members), is plagiarism.

Asking a third party to complete coursework on your behalf, and then submitting it as your own work, is contract cheating.

You must never give a copy of your coursework writing or code to another student (other than the group members), and you must always refuse any request from another student for a copy of your coursework. Sharing your coursework with another student (other than the group members) is collusion.

Suspected plagiarism, contract cheating or collusion will be referred to the School Discipline Committee for investigation. This will delay your feedback, and if you are found guilty of misconduct then it will result in a penalty such as voiding the course or being withdrawn from the university.

The use of AI generation and code completion tools is not acceptable.

So please don't do any of the things above! If you have any concerns about potential academic misconduct, please ask your local course leader for guidance.

For details see the University's academic misconduct policy and this Student Guide to Academic Integrity.